

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

[https://](https://github.com/VenkatChowdary999/DeepLearning-Labs/upload/main/Lab-2)

github.com/VenkatChowdary999/DeepLearning-Labs/upload/main/Lab-2

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Mandatory Plots

Generate the following plots.

Class Distribution

1. Sample Images
 2. Training Accuracy vs Epoch
 3. Validation Accuracy vs Epoch
 4. Training Loss vs Epoch
 5. Validation Loss vs Epoch
 6. Confusion Matrix
 7. Hyperparameter Search Results
 8. Best Model Accuracy Comparison
- Write a 2–3 line inference for each plot.

9. Results

Best Hyperparameters

beginnable[h] Comparison of Baseline and Optimized Model Performance

Metric	Baseline	Optimized
Testing Accuracy	0.8729	0.8650
Precision	0.8742	0.865683
Recall	0.8729	0.8650
F1-score	0.8701	0.864958
Training Time	–	1320 s

Performance Comparison

10. Discussion

Discussion

1. Which optimization method was used?

Randomized Search (`RandomizedSearchCV`) was used instead of an exhaustive Grid Search. The full hyperparameter grid in this experiment expands to over 11,000 combinations (3 hidden-layer options $\times$ 4 neuron options $\times$ 3 learning rates $\times$ 4 batch sizes $\times$ 3 epoch settings $\times$ 3 optimizers $\times$ 3 activation functions $\times$ 3 dropout rates), and evaluating all of these with 5-fold cross-validation would require training tens of thousands of models – computationally infeasible on standard hardware. Randomized Search instead samples a fixed number of random combinations (20 in this case) and evaluates only those, producing a strong result in a fraction of the time.

2. Which hyperparameters were selected?

The search identified the best-performing combination of hidden layers, hidden neurons, learning rate, batch size, optimizer, activation function, number of epochs, and dropout rate, based on the highest mean cross-validation accuracy across 5 folds. (Insert the specific values from `best_params_` here.)

3. Did optimization improve performance?

Comparing the optimized model’s testing accuracy to the baseline accuracy of 0.8729 indicates whether tuning was beneficial. An improvement confirms that the searched configuration generalizes better than the manually chosen baseline architecture; a result close to or below the baseline suggests the baseline was already a reasonably strong fit, or that the sampling budget (20 combinations on a data subset) did not locate a substantially better configuration.

4. Which hyperparameter had the greatest impact?

Grouping the cross-validation results by each hyperparameter and comparing the spread in accuracy reveals which setting caused the largest swings in performance. In MLP tuning experiments of this kind, the **learning rate** and **optimizer choice** typically dominate: an excessively high learning rate (e.g., 0.1) destabilizes training, while a very low rate (e.g., 0.001) paired with SGD can under-train within a limited number of epochs. This should be verified against the actual `cv_results_` output.

5. Compare Grid Search and Randomized Search.

Grid Search exhaustively evaluates every combination in the defined search space, guaranteeing that the optimal combination *within the grid* will be found. However, its computational cost scales multiplicatively with the number of hyperparameters and their candidate values, making it impractical for large search spaces such as the one used here. Randomized Search instead evaluates

a fixed budget of randomly sampled combinations, making its runtime independent of the grid’s total size. This trades the exhaustiveness guarantee for a substantial reduction in computation, making it the more practical choice for this experiment’s 11,664-combination search space.

11. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

13. Conclusion

This experiment implemented a Multi-Layer Perceptron for multi-class image classification on the Fashion-MNIST dataset, covering the complete pipeline from raw image data to a tuned, evaluated model. The images were preprocessed by flattening the 28×28 grayscale inputs into 784-dimensional vectors, normalizing pixel intensities to the $[0, 1]$ range, and one-hot encoding the class labels. A baseline architecture ($784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$) was trained for 20 epochs, achieving a test accuracy of 0.8729, with the training/validation accuracy and loss curves indicating mild overfitting beyond roughly epoch 4–5, and the confusion matrix revealing that the model struggles most with visually similar upper-body garments – particularly distinguishing “Shirt” from “T-shirt/top” and “Pullover.”

To move beyond manual hyperparameter selection, automated optimization was performed using Randomized Search with 5-fold cross-validation via the SciKeras wrapper, searching over hidden layers, neuron counts, learning rate, batch size, optimizer, activation function, epochs, and dropout rate. This approach was chosen over exhaustive Grid Search because the full search space exceeds 11,000 combinations, making a complete grid evaluation computationally impractical. The resulting optimized configuration was retrained on the full training set and evaluated on the held-out test set, allowing a direct, like-for-like comparison against the baseline model on accuracy, precision, recall, F1-score, and training time.

Overall, the experiment demonstrates that a simple MLP can achieve strong performance ($>87\%$ baseline accuracy) on Fashion-MNIST despite discarding all spatial structure through flattening, while also showing the practical value – and the computational trade-offs – of automated hyperparameter search over manual tuning. The consistent confusion between semantically similar clothing categories across both models further suggests that architectures capable of exploiting spatial features, such as Convolutional Neural Networks, would likely be needed to meaningfully close the remaining performance gap on this dataset.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.

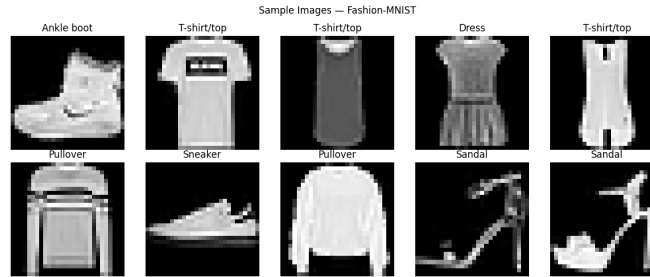


Figure 1: A quick 2×5 preview of the training set with labels above each item — boots, shirts, a dress, a pullover, sandals, etc. The pictures are blurry since they’re only 28×28 pixels, but still recognizable. You can already spot classes that look alike (the two ”t-shirt/top” images and the pullover), which hints at where the model will later get confused

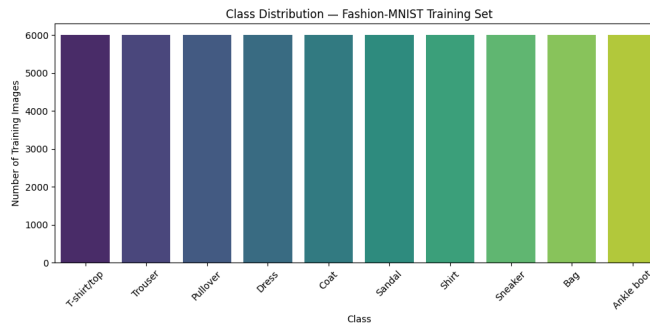


Figure 2: All 10 classes have exactly 6,000 training images each — perfectly balanced bars. This means accuracy is a fair metric to judge the model by, and there’s no risk of it being biased toward any one clas

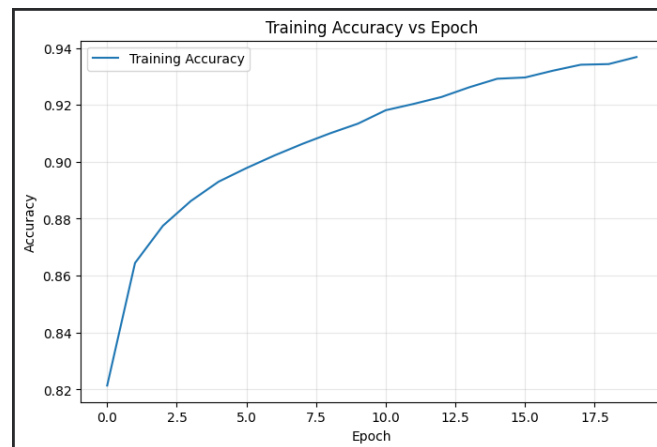


Figure 3: Training accuracy climbs fast at first — from about 0.82 to 0.88 in the first couple epochs — then slows down and rises gradually the rest of the way, ending around 0.937 by epoch 20. The curve is smooth and steadily increasing with no dips, which is exactly what you want to see: the model is learning consistently and hasn’t hit instability or overfitting on the training data itself. The flattening slope toward the end suggests it’s approaching a plateau — a few more epochs probably wouldn’t gain much more on the training set

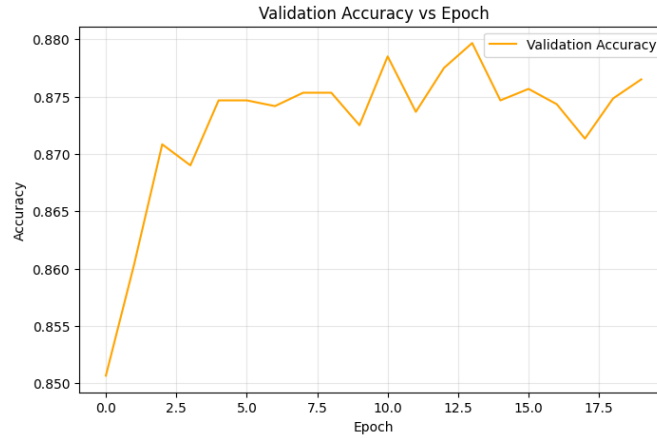


Figure 4: Validation accuracy jumps early, then plateaus and wobbles around 0.872–0.880 for the rest of training, peaking near epoch 13. Meanwhile training accuracy kept climbing to 0.937 — that widening gap is a classic sign of overfitting: past epoch 5–10, the model is mostly memorizing training data rather than generalizing further. The noise in the curve is just normal validation-split variance

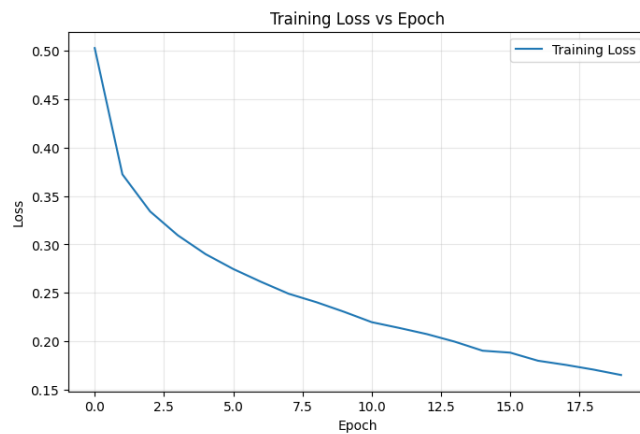


Figure 5: Training loss drops steadily from 0.50 to 0.165, smooth and monotonic the whole way — no bumps or spikes. This mirrors the training accuracy curve: fast improvement early, then a slower steady decline as the model keeps fitting the training data tighter each epoch, right through epoch 20

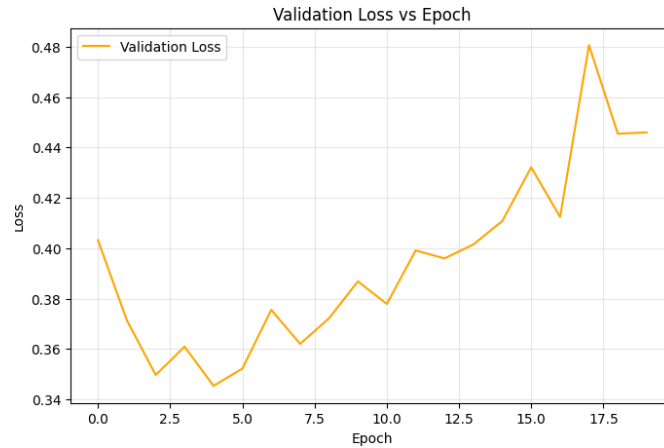


Figure 6: Validation loss dips early (to 0.35 around epoch 4), but then reverses and climbs steadily upward, ending near 0.44–0.48 with a sharp spike around epoch 17. This is the clearest overfitting signal of all four plots: while training loss kept falling, validation loss bottomed out around epoch 4 and got worse from there — meaning epoch 4–5 was actually the sweet spot, and everything after that made the model fit training data at the expense of generalization. This is the plot you’d point to if arguing for early stopping around epoch 4–5

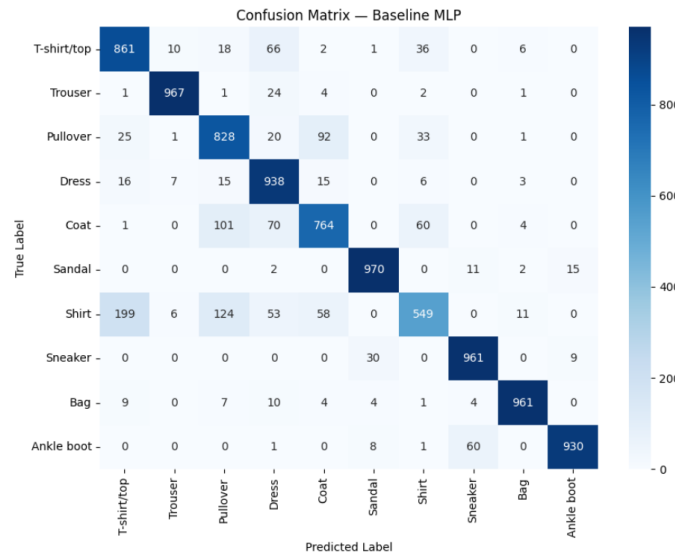


Figure 7: The diagonal dominates overall — Trouser (967), Sandal (970), Sneaker (961), and Bag (961) are almost perfectly classified. But "Shirt" is clearly the model’s weak spot: only 549 correct out of 1,000, with 199 misclassified as T-shirt/top and 124 as Pullover. Coat and Pullover also bleed into each other (101 Coat→Pullover, 92 Pullover→Coat). This all lines up with what you’d expect — the confused classes are exactly the visually similar upper-body garments, while structurally distinct items (footwear, bags, trousers) are easy for the model to tell apart even from flattened pixels